Pub/Sub Consumer Guidelines

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

Pub/Sub is the backbone of our asynchronous flows: ingestion-service publishes, worker-service and scheduler consume. Its delivery contract is **at-least-once**, not exactly-once-by-default, which trips up teams who assume a message arrives exactly once and in order. This page is the prescriptive how-to for writing a consumer that survives redelivery, slow processing, and errors. It exists to prevent duplicated side effects (BUG-1031), redelivery storms from a too-short ack deadline (BUG-1032), and messages stuck unacked because the handler swallows errors (BUG-1033).

Standards

Make every consumer idempotent

Because Pub/Sub redelivers, a handler *will* run twice for some messages. Never assume single delivery. Dedupe on a stable key — either the Pub/Sub `message_id` or, better, a business key from the payload — and record it in the *same* transaction as the side effect so the check and the write cannot diverge. For strict cases, enable exactly-once delivery on the subscription, but keep the idempotency guard regardless.

```
def handle(message):
   key = message.attributes["event_id"]  # stable business key
   with spanner_db.run_in_transaction as txn:  # dedupe + effect atomically
       if txn.exists("ProcessedEvents", key):
           message.ack()          # already done; drop the duplicate
           return
       apply_side_effect(txn, message.data)
       txn.insert("ProcessedEvents", key)
   message.ack()
```

Ack only on success; nack on failure

Ack means "done, never send again." Ack *after* the work succeeds, and ack only then. On failure, nack (or let the exception propagate so the client nacks) so the message is retried promptly instead of silently sitting unacked until the deadline expires every cycle — the stuck-backlog pattern in BUG-1033. Route messages that fail repeatedly to a dead-letter topic so one bad message never blocks the stream.

```
def callback(message):
   try:
       handle(message)           # acks internally on success
   except Exception:
       logging.exception("processing failed for %s", message.message_id)
       message.nack()            # retry now, don't wait for the deadline
```

```
# Dead-letter after N attempts so poison messages don't loop forever
gcloud pubsub subscriptions update worker-sub \
 --dead-letter-topic=events-dlq \
 --max-delivery-attempts=5
```

Size the ack deadline and concurrency to the work

If processing takes longer than the subscription's ack deadline, Pub/Sub assumes failure and redelivers — each redelivery adds more in-flight work until the backlog explodes (BUG-1032). Set an ack deadline above your p99 processing time, let the client library extend the lease automatically for long tasks, keep per-message work small, and raise flow-control concurrency so messages are acked before the deadline.

```
from google.cloud import pubsub_v1

flow = pubsub_v1.types.FlowControl(max_messages=200)     # bounded concurrency
subscriber.subscribe(sub_path, callback=callback, flow_control=flow)

# Ack deadline > p99 processing time; client also auto-extends the lease
gcloud pubsub subscriptions update worker-sub --ack-deadline=120
```

Prefer moving heavy work off the callback thread (a bounded worker pool) so the subscriber keeps leasing and extending deadlines instead of stalling.

Common failures and fixes

- Downstream side effects happen twice → dedupe on a business/message key stored in the same transaction as the effect, or enable exactly-once delivery (BUG-1031).
- Backlog grows with constant redelivery → raise the ack deadline above p99, rely on automatic lease extension, shrink per-message work, and increase concurrency (BUG-1032).
- Unacked count climbs and messages are "stuck" → nack on error and ack only on success; never return from the handler without acking or nacking (BUG-1033).
- One malformed message loops forever → configure a dead-letter topic with a sane `max-delivery-attempts` and alert on DLQ volume.

Checklist

- Handler is idempotent; dedupe key recorded transactionally with the side effect.
- Ack happens only after success; failures nack (or propagate) so retry is prompt.
- Ack deadline exceeds measured p99 processing time; lease auto-extension enabled.
- Flow-control concurrency tuned; heavy work offloaded from the callback thread.
- Dead-letter topic configured with `max-delivery-attempts` and alerting.
- Consumer does not assume ordering unless an ordering key is set on publish.

Related incidents

BUG-1031, BUG-1032, BUG-1033